

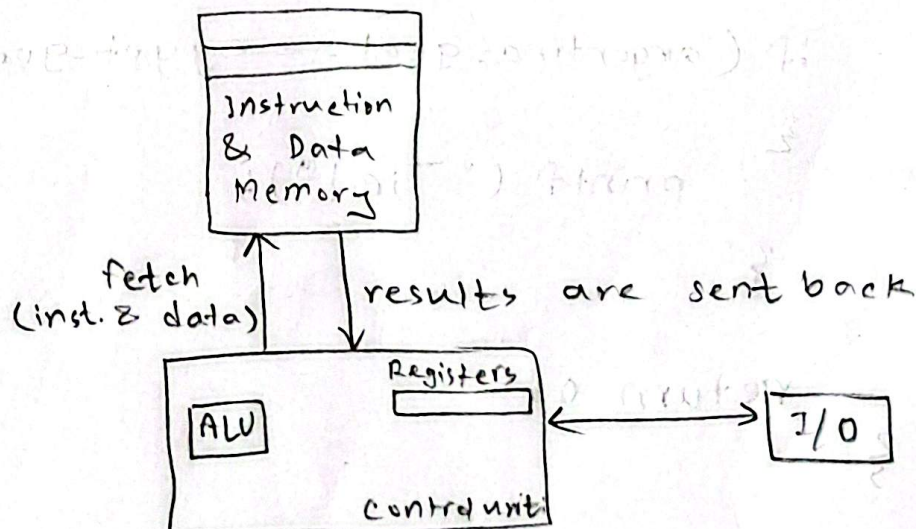
Name: Aziza Zaman Sadia ; ID: 2231603642
Course & Section: CSE425.1

Ans. to the ques. no-01

Imperative programming language implies how to achieve a result using step by step commands and state changes. They focus heavily on how an algorithm should execute step-by-step.

As far as Von-Neumann architecture goes, it physically separated CPU from memory component for the first time. Moreover, the memory compartment held both data and instruction. So, imperative programming directly mirrors Von-Neumann physical hardware. Variables in code map directly to physical cells in memory unit, and assignment statements function as data movement lines piping data between memory cells and CPU registers.

The schematic diagram of Von-Neumann architecture would be.



Ans. to the ques. no. 02

In C programming, if-statement is not mandatory, given that while statement is available. Both are control flow statements, an example would be,

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int argentina-goal = 3;
```

```
    int egypt-goal = 2;
```

```
    if (argentina-goal > egypt-goal)
```

```
    {
```

```
        printf("Argentina won!");
```

```
    }
```

```
    if (argentina-goal < egypt-goal)
```

```
    {
```

```
        printf("Egypt won!");
```

```
    }
```

```
    if (argentina-goal == egypt-goal)
```

```
    {
```

```
        printf("Tie!");
```

```
    }
```

```
    return 0;
```

```
}
```

Replacing if-statements with while-statements,

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int argentina-goal = 3;
```

```
    int egypt-goal = 2;
```

```
    while (argentina-goal > egypt-goal)
```

```
    {
```

```
        printf("Argentina won!");
```

```
        break;
```

```
    }
```

```
    while (argentina-goal < egypt-goal)
```

```
    {
```

```
        printf("Egypt won!");
```

```
        break;
```

```
    }
```

```
    while (argentina-goal == egypt-goal)
```

```
    {
```

```
        printf("Tie!");
```

```
        break;
```

```
    }
```

```
    return 0;
```

```
}
```



Start here x if-while.c x

```
1 #include<stdio.h>
2
3 int main()
4 {
5     int aregentina_goal = 3;
6     int egypt_goal = 2;
7
8     if(aregentina_goal > egypt_goal)
9     {
10         printf("Argentina won!\n");
11     }
12     if(aregentina_goal < egypt_goal)
13     {
14         printf("Egypt won!\n");
15     }
16     if(aregentina_goal == egypt_goal)
17     {
18         printf("Tie!\n");
19     }
20
21     printf("\n\n\n\nNow, here are the results of the while statements.\n");
22
23     while(aregentina_goal > egypt_goal)
24     {
25         printf("Argentina won!\n");
26         break;
27     }
28     while(aregentina_goal < egypt_goal)
29     {
30         printf("Egypt won!\n");
31         break;
32     }
33     while(aregentina_goal == egypt_goal)
34     {
35         printf("Tie!\n");
36         break;
37     }
38
39 }
```

C:\Users\Hp\Downloads\if-wf x

Argentina won!

Now, here are the results of the while statements.
Argentina won!

Process returned 0 (0x0) execution time : 0.081 s
Press any key to continue.

Logs & others

Code::Blocks x Search results x Cccc x Build log x Build messages x CppCheck/Vera++ x CppCheck/Vera++ messages x Cscope x Debugger x DoxyBlocks x Fortran info x Closed files list x

Ans. to the ques no-03

Generality $\Rightarrow$

It implies avoiding special context restrictions or exceptional rules. Similar rules should apply uniformly to all relevant constructs without creating exceptions to the rule.

The language in choice is C. C has poor generality qualities. Nested functions are not allowed in C and '=' equality operator can be used to perform equality operation on integers, floats, characters but don't work for string comparison.

Extensibility $\Rightarrow$

It implies providing an easy mechanism so that new - database libraries, functions to libraries - can be added comfortably. (without breaking or rewriting the core compiler).

Sometimes, even though languages provide built-in primitive data types, we need to create new data types all the time by defining objects/classes.

In C, we can do this by using typedef struct. An example would be,

```
typedef struct Node
```

```
{
```

```
    int data;
```

```
    struct Node *Next;
```

```
} Node;
```

Uniformity ⇒

It implies achieving consistency in the appearance and behaviour of language constructs.

For C, after functions we don't have to end with a semicolon but for defining structures, we have to put a semicolon after that - which is inconsistent in the appearance.

Restrictability ⇒

It implies intentionally placing boundaries on powerful features to safeguard application reliability and stop silent memory exploits.

In C, we have strict structural limitations. We can't dynamically resize arrays but unless a safe transition to the heap.

Ans. to the ques. no - 04

The four types of errors - that generally occur in a program are -

① Lexical Error ⇒

It occurs when the compiler reads raw text, it flags unfamiliar or illegal characters.

② Syntactical Error ⇒

These represent structural errors. When code violates grammatical rules it flags that. By grammar constraints it is meant to be the language's BNF.

③ Logical Errors ⇒

When code compiles and runs without crashing, but produces the wrong outputs.

④ Runtime Exception $\Rightarrow$

These kind of error occurs when the program is actively running. It occurs due to zero or null pointer exception. It halts the execution pipeline dynamically mid-run.

Ans. to the ques. no-05

a

Compilers are separated into front-end and backend because of the difference in machine dependency.

Frontend is platform independent but the backend is platform-dependent.

b

There is a big bottleneck in Von-Neumann architecture - as there is only one memory component holding both data and instructions, there is only one bus connecting memory to CPU.

That means CPU speed is exponentially faster than the data memory bus. The CPU sends idle cycles waiting for memory transfers.

C
The roles Symbol Table have in compilers -

→ catalogs every variable identifier name, function label, and static type discovered in the code,

→ tracks scope, memory footprints.

D
Interpreter-based implementation and the hybrid approach holds portability properties. The capacity of a programming language's source code to compile and execute across diverse operating systems and

- CPU hardware environments without requiring rewrites.

Ans. to the ques. no-06

From the example,

```
#include <stdio.h>
int i = 6;  $\Rightarrow$  This is a global variable
int main (int args, char *argv[])
{
    printf("%d\n", i);  $\Rightarrow$  no local 'i' declared yet,
     $\Rightarrow$  This is the start of a local block
    {
        int i = 7, j = 8;
        printf("%d\n", i, j);  $\Rightarrow$  outputs 7, 8
         $\Rightarrow$  This is the start of another local block.
        {
            int i = 5;
            printf("%d\n", i, j);  $\Rightarrow$  outputs
            5, 8
        }
        printf("%d\n", i);  $\Rightarrow$  outputs 7
    }
    printf("%d\n", i);  $\Rightarrow$  outputs 6
    return 0;
}
```

We use static scoping - which binds references at compile time based strictly on spatial layout blocks.

As there are two isolated inner blocks in the piece of code, the value of 'i' changes because of the stack memory frame.

static-blocks.c - Code::Blocks 25.03

FileEditViewSearchProjectBuildDebugFortranwxSmithToolsTools+PluginsDoxyBlocksSettingsHelp

<global>

Start here X static-blocks.c X

```
1 #include <stdio.h>
2
3 int i = 6;
4
5 int main(int argc, char *argv[])
6 {
7     printf("%d\n", i);
8     {
9         int i = 7, j = 8;
10        printf("%d\n%d\n", i, j);
11        {
12            int i = 5;
13            printf("%d\n%d\n", i, j);
14        }
15        printf("%d\n", i);
16    }
17    printf("%d\n", i); /
18    return 0;
19 }
20
```

Logs & others

Code::Blocks X Search results X Cccc X Build log X Build messages X CppCheck/Vera++ X CppCheck/Vera++ messages X

File	Line	Message
		=== Build file: "no target" in "no project" (compiler: unknown) ===
		=== Build finished: 0 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===

C:\Users\Hp\Downloads\stati X + v - □ ×

6
7
8
5
8
7
6

Process returned 0 (0x0) execution time : 0.092 s
Press any key to continue.

Ans. to the ques. no-07

• Shortcode $\Rightarrow$

Description $\Rightarrow$ This was designed for binary calculation processing.

Advantages $\Rightarrow$ This offered an upgrade over writing absolute raw binary machine code.

Disadvantages $\Rightarrow$ It was purely interpreted - as it had to translate each line of instruction on the fly - it resulted in a very slow execution speed.

• ALGOL 58-60 $\Rightarrow$

Description $\Rightarrow$ It was designed by an international committee to formalize arithmetic, algorithmic logic.

Advantages $\Rightarrow$ Popularized block structures, explicit variable type declaration.

Shortcomings $\Rightarrow$ It was too orthogonal affecting the readability and writability for the programmers.

• Python $\Rightarrow$

Description $\Rightarrow$ High-level dynamically typed language that prioritizes readability, writeability.

Advantages $\Rightarrow$ Features highly expressive syntax and minimal boilerplate code, has great extensibility features.

Shortcomings $\Rightarrow$ Dynamically typed variables slow execution.

• Assembly Languages $\Rightarrow$

Description $\Rightarrow$ Low-level, symbolic language, that later gets translated to machine code.

Advantages $\Rightarrow$ provides direct access to hardware registers, memory layouts.

Shortcomings $\Rightarrow$ This is platform-dependent

• C/C++ ⇒

Description ⇒ structured system-level imperative programming was the main aspect of C and C++ offered object-oriented features and abstractions on top of that.

Advantages ⇒ raw, low-pointer access and direct compilation

Shortcomings ⇒ lacks automatic memory safety.

• Fortran ⇒

Description ⇒ This was created by John Backus for numerical calculations. It was highlighted for its execution speed.

Advantages ⇒ greatly reduced development time, highlighted execution speed.

Shortcomings $\Rightarrow$ Lacked advanced data structures and string-handling capabilities

• Java $\Rightarrow$

Description $\Rightarrow$ class-based, object-oriented hybrid language.

Advantages $\Rightarrow$ Implemented "Write once, run anywhere".

Shortcomings $\Rightarrow$ virtual machine layer adds memory overhead and runtime delays compared to languages that compile directly.

• LISP / Scheme $\Rightarrow$

Description $\Rightarrow$ Foundational functional programming language.

Advantages $\Rightarrow$ powerful recursion architectures, dynamic memory management.

Shortcomings $\Rightarrow$ heavily nested syntax
affected readability and track
compared to standard procedural
layouts.